CIÊNCIAS EXATAS E DA TERRA

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS - TECNÓLOGO

LUCAS ANTONIO DA SILVA

# PORTIFÓLIO INDIVIDUAL:

Projeto Integrado Síntese

São Paulo
2024

LUCAS ANTONIO DA SILVA

# PORTIFÓLIO INDIVIDUAL:
Projeto Integrado Síntese

Trabalho de portfólio apresentado como requisito parcial para a obtenção de média semestral.

Orientadores: Alexandro e Rogian Villa

São Paulo
2024

**SUMÁRIO**


1. **INTRODUÇÃO** ..... **3**
2. **MÉTODOS** ..... **4**
3. **RESULTADOS** ..... Erro! Indicador não definido.
4. **CONCLUSÃO** ..... **13**
5. **REFERÊNCIAS** ..... **14**

# 1. INTRODUÇÃO

O gerenciamento eficiente de estoque é um desafio comum para empresas em rápido crescimento, que precisam lidar com problemas como falta de produtos, excesso de estoque e rastreamento de itens. Este trabalho propõe o desenvolvimento de um sistema de gestão de estoque para otimizar essas operações, garantindo maior precisão e agilidade na gestão dos produtos. As funcionalidades do sistema abrangem o cadastro de produtos, atualização de estoque, rastreamento de localização dos itens, e geração de relatórios detalhados, atendendo às necessidades de organização e controle da empresa. Utilizando uma abordagem baseada em metodologia ágil (Scrum), lógica booleana e modelagem de casos de uso, o projeto busca melhorar a experiência e eficiência do gerenciamento de estoque.

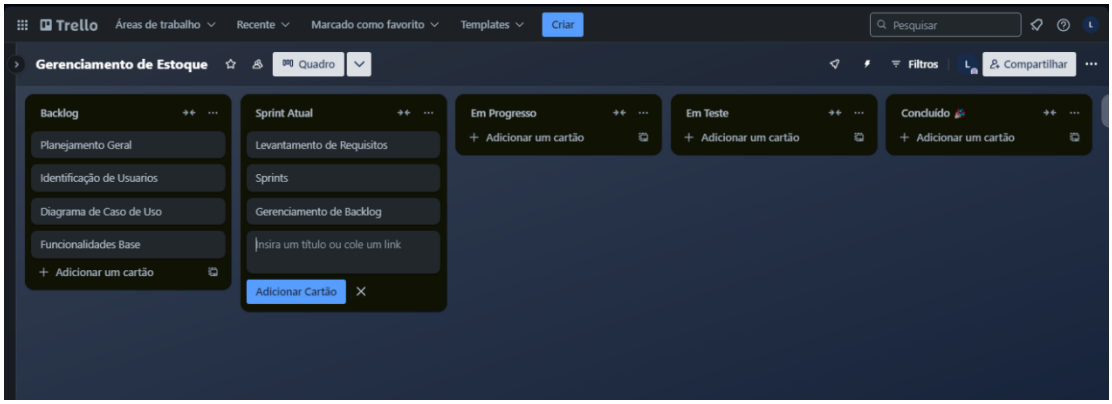
## 2. MÉTODOS E RESULTADOS

Para o desenvolvimento do sistema, utilizou-se uma abordagem estruturada em quatro fases principais:

- Planejamento e Organização;
- Desenvolvimento de Estrutura e Algoritmos;
- Modelagem Lógica;
- Modelagem de Casos de Uso;

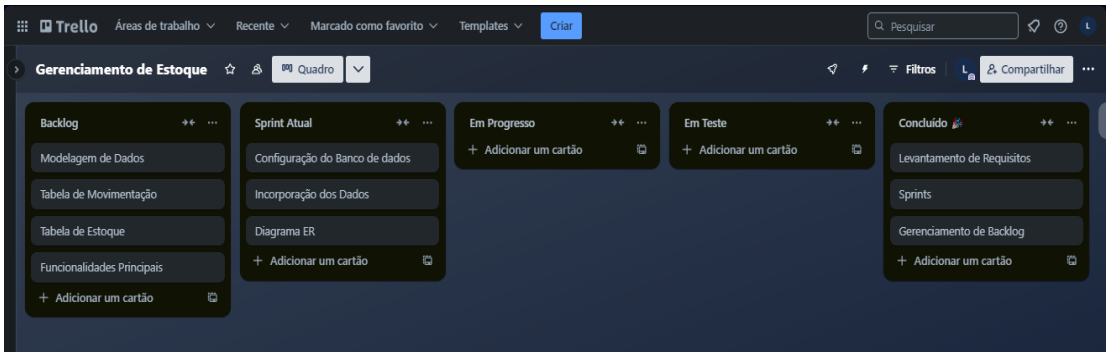
**2.1. Planejamento e Organização (Scrum)**: Um quadro Scrum foi criado no Trello, com listas para organizar o backlog, tarefas em andamento e concluídas. As sprints foram planejadas semanalmente para alcançar marcos específicos do sistema.

### Sprint 1: Planejamento



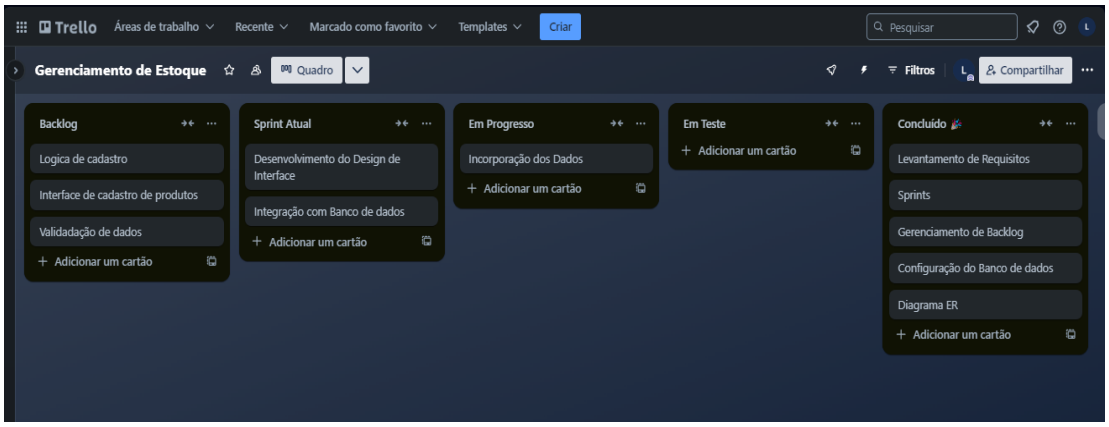

Essa sprint tem como objetivo definir e planejar os requisitos do sistema a ser desenvolvido em um período estimado de 2 semanas.

## Sprint 2: Modelagem de Dados



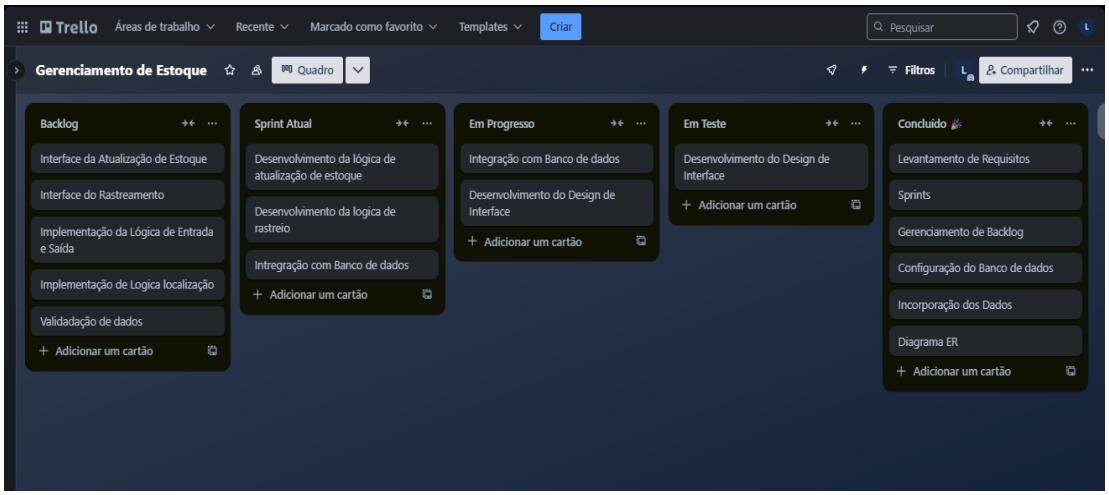

Essa Sprint foi pensada para definir a estrutura de dados e moldar o banco de dados em um prazo de 2 semanas.

## Sprint 3: Desenvolvimento do Cadastro de Produtos



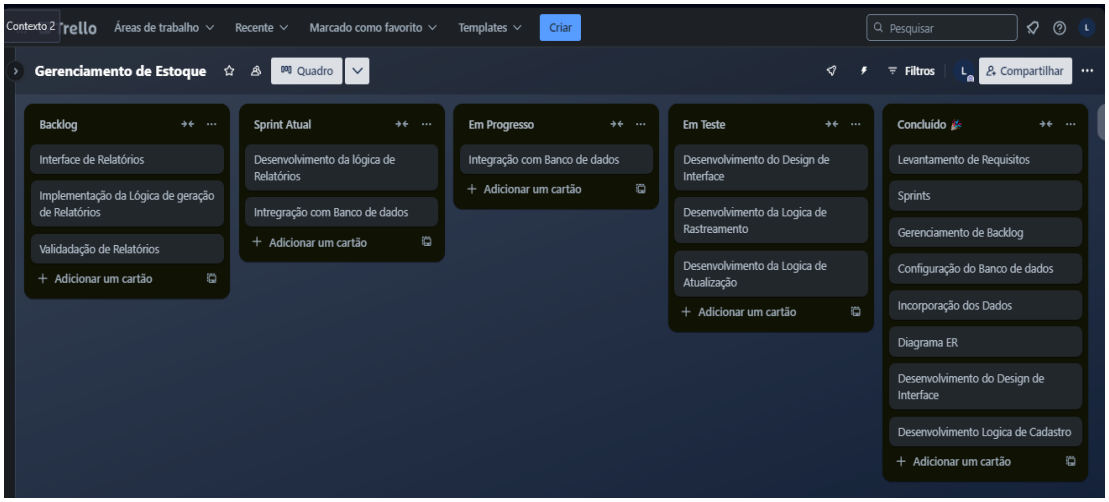

O desenvolvimento dessa Sprint foi focado na implementação do cadastro de produtos no período de até 3 semanas.

## Sprint 4: Desenvolvimento da Atualização de Estoque e Rastreamento



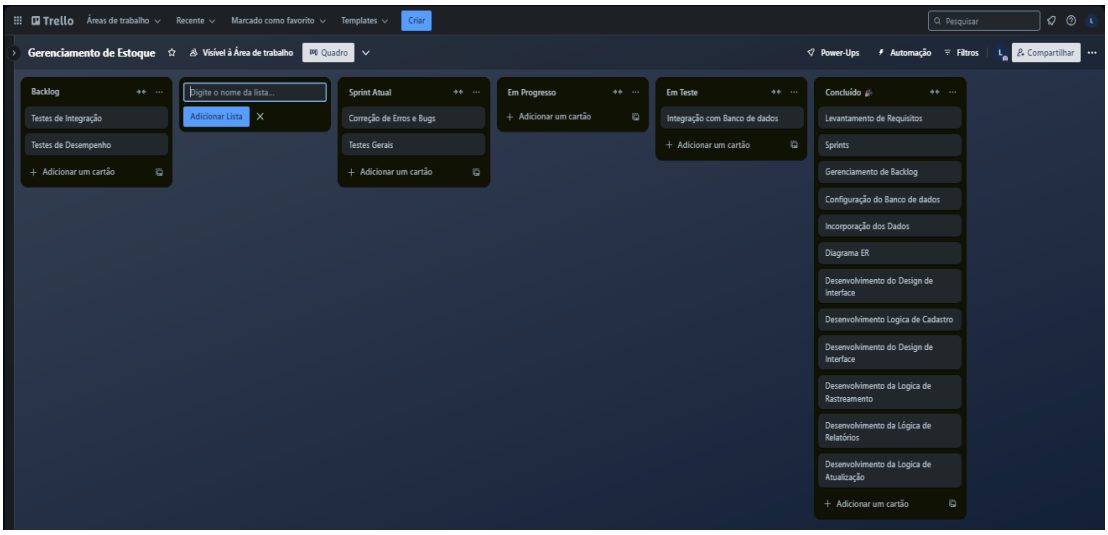

Nessa Sprint foi definido o objetivo de implementar a atualização de estoque e rastreamento de localização em um período estimado de 5 semanas.

## Sprint 5: Desenvolvimento de Relatórios




Essa Sprint teve como objetivo implementar o módulo de geração de relatórios em um período de 3 semanas.

## Sprint 6: Testes Finais




Ao decorrer dessa Sprint de 2 semanas foram realizados os testes finais e a implementação do sistema

**2.2. Desenvolvimento de Estruturas e Algoritmos**: Foram desenvolvidas estruturas de dados para cadastro de produtos, categorias e movimentações de estoque, além de algoritmos que possibilitam o cadastro e atualização de produtos, consulta ao estoque, geração de relatórios e rastreamento de localização.

- **Estrutura de Dados**

```python
# Estrutura de dados para Produtos
class Produto:
    def __init__(self, cod_produto, nome, categoria, quantidade, preco, localizacao):
        self.cod_produto = cod_produto
        self.nome = nome
        self.categoria = categoria
        self.quantidade = quantidade
        self.preco = preco
        self.localizacao = localizacao

# Estrutura de dados para Categoria
class Categoria:
    def __init__(self, cod_categoria, nome):
        self.cod_categoria = cod_categoria
        self.nome = nome

# Estrutura de dados para Movimentação de Estoque
class Movimentacao:
    def __init__(self, cod_produto, cod_categoria, data, quantidade, tipo):
        self.cod_produto = cod_produto
        self.cod_categoria = cod_categoria
        self.quantidade = quantidade
        self.tipo = tipo  # 'entrada' ou 'saida'
        self.data = data
```

- **Algoritmos de Cadastro e Consulta**

```python
# Dicionários para armazenar dados
produtos = {}
categorias = {}
movimentacoes = []

# Contadores para IDs automáticos
cod_produto_counter = 1
cod_categoria_counter = 1
cod_movimentacao_counter = 1

# Função para cadastrar categoria
def cadastrar_categoria(nome):
    global cod_categoria_counter
    categoria = Categoria(cod_categoria_counter, nome)
    categorias[cod_categoria_counter] = categoria
    cod_categoria_counter += 1
    return categoria

# Função para cadastrar produto
def cadastrar_produto(nome, cod_categoria, quantidade, preco, localizacao):
    global cod_produto_counter
    if cod_categoria not in categorias:
        raise ValueError("Categoria não encontrada.")
    produto = Produto(cod_produto_counter, nome, cod_categoria, quantidade, preco, localizacao)
    produtos[cod_produto_counter] = produto
    cod_produto_counter += 1
    return produto

# Função para consultar produto
def consultar_produto(cod_produto):
    return produtos.get(cod_produto, "Produto não encontrado")
```

- **Algoritmos de Movimentação**

```python
# Função para registrar movimentação de estoque
def registrar_movimentacao(cod_produto, tipo, quantidade, data):
    global cod_movimentacao_counter
    if cod_produto not in produtos:
        raise ValueError("Produto não encontrado.")
    if tipo not in ['entrada', 'saida']:
        raise ValueError("Tipo de movimentação inválido.")

    produto = produtos[cod_produto]
    if tipo == 'saida' and produto.quantidade < quantidade:
        raise ValueError("Quantidade insuficiente em estoque.")

    if tipo == 'entrada':
        produto.quantidade += quantidade
    elif tipo == 'saida':
        produto.quantidade -= quantidade

    # Registrar movimentação
    movimentacao = Movimentacao(cod_produto, produto.categoria, data, quantidade, tipo)
    movimentacoes.append(movimentacao)
    cod_movimentacao_counter += 1
    return movimentacao
```

- **Relatórios  e Consultas**

```python
# Relatório e Consultas
def gerar_relatorio_estoque():
    relatorio = []
    for id_produto, produto in produtos.items():
        relatorio.append({
            'id_produto': id_produto,
            'nome': produto.nome,
            'categoria': categorias[produto.categoria].nome,
            'quantidade': produto.quantidade,
            'preco': produto.preco,
            'localizacao': produto.localizacao
        })
    return relatorio

def consulta_movimentacoes(id_produto):
    if id_produto not in produtos:
        raise ValueError("Produto não encontrado.")
    historico = [mov for mov in movimentacoes if mov.cod_produto == id_produto]
    return historico

def gerar_relatorio_movimentacoes():
    relatorio = []
    for mov in movimentacoes:
        relatorio.append({
            'id_produto': mov.cod_produto,
            'nome_produto': produtos[mov.cod_produto].nome,
            'tipo': mov.tipo,
            'quantidade': mov.quantidade,
            'data': mov.data
        })
    return relatorio
```

- **Testes do Código**

```python
# 1. Cadastrar categorias
cat1 = cadastrar_categoria("Eletrônicos")
cat2 = cadastrar_categoria("Móveis")
print("Categorias cadastradas:")
print(categorias)

# 2. Cadastrar produtos
prod1 = cadastrar_produto("Notebook", cat1.cod_categoria, 10, 3000.0, "Estante A1")
prod2 = cadastrar_produto("Mesa", cat2.cod_categoria, 5, 500.0, "Estante B2")
print("\nProdutos cadastrados:")
print(produtos)

# 3. Consultar um produto específico
print("\nConsulta de produto:")
print(consultar_produto(prod1.cod_produto))

# 4. Registrar movimentação de entrada e saída de estoque
from datetime import datetime

# Movimentação de entrada
mov1 = registrar_movimentacao(prod1.cod_produto, "entrada", 5, datetime.now())
# Movimentação de saída
mov2 = registrar_movimentacao(prod1.cod_produto, "saida", 3, datetime.now())
print("\nMovimentações registradas:")
for mov in movimentacoes:
    print(mov.__dict__)

# 5. Gerar relatório de estoque
print("\nRelatório de Estoque:")
estoque = gerar_relatorio_estoque()
for item in estoque:
    print(item)
# 6. Gerar histórico de movimentações para um produto específico
print("\nHistórico de movimentações para o produto", prod1.cod_produto)
historico = consulta_movimentacoes(prod1.cod_produto)
for mov in historico:
    print(mov.__dict__)

# 7. Gerar relatório geral de movimentações
print("\nRelatório de Movimentações:")
relatorio_mov = gerar_relatorio_movimentacoes()
for item in relatorio_mov:
    print(item)
```

**2.3.** **Modelagem Lógica**: Uma tabela verdade foi criada para representar as variáveis booleanas que indicam o atendimento de cada requisito (Cadastro de Produtos, Atualização de Estoque, Rastreamento de Localização e Relatórios).

- **Variáveis Booleanas:**

  **P: Cadastro de Produtos**
  **E: Atualização de Estoque**
  **L: Rastreamento de Localização**
  **R: Relatórios**

**Expressão Lógica**: A condição para todos os requisitos serem atendidos é: P∧ E ∧ L ∧ R. A "Solução Completa" será verdadeira apenas se todos os valores forem verdadeiros.

| P | E | L | R | Solução Completa |
|---|---|---|---|---|
| T | T | T | T | T |
| T | T | T | F | F |
| –T | T | F | T | F |
| T | T | F | F | F |
| T | F | T | T | F |
| T | F | T | F | F |
| T | F | F | T | F |
| T | F | F | F | F |
| F | T | T | T | F |
| F | T | T | F | F |
| F | T | F | T | F |
| F | T | F | F | F |
| F | F | T | T | F |
| F | F | T | F | F |
| F | F | F | T | F |
| F | F | F | F | F |

- **P (Produtos):** True (T) se o sistema permite cadastro de novos produtos.
- **E (Estoque):** True (T) se o sistema permite a atualização de quantidade de produtos em estoque.
- **L (Localização):** True (T) se o sistema permite o rastreio da localização dos produtos.
- **R (Relatórios):** True (T) se p sistema gera relatórios sobre o status do estoque.

**2.4. Modelagem de Casos de Uso**: Utilizou-se um diagrama de casos de uso para representar as interações entre os tipos de usuários do sistema (Estoquista, Usuário e Gerente de Setor) e suas respectivas permissões e ações.

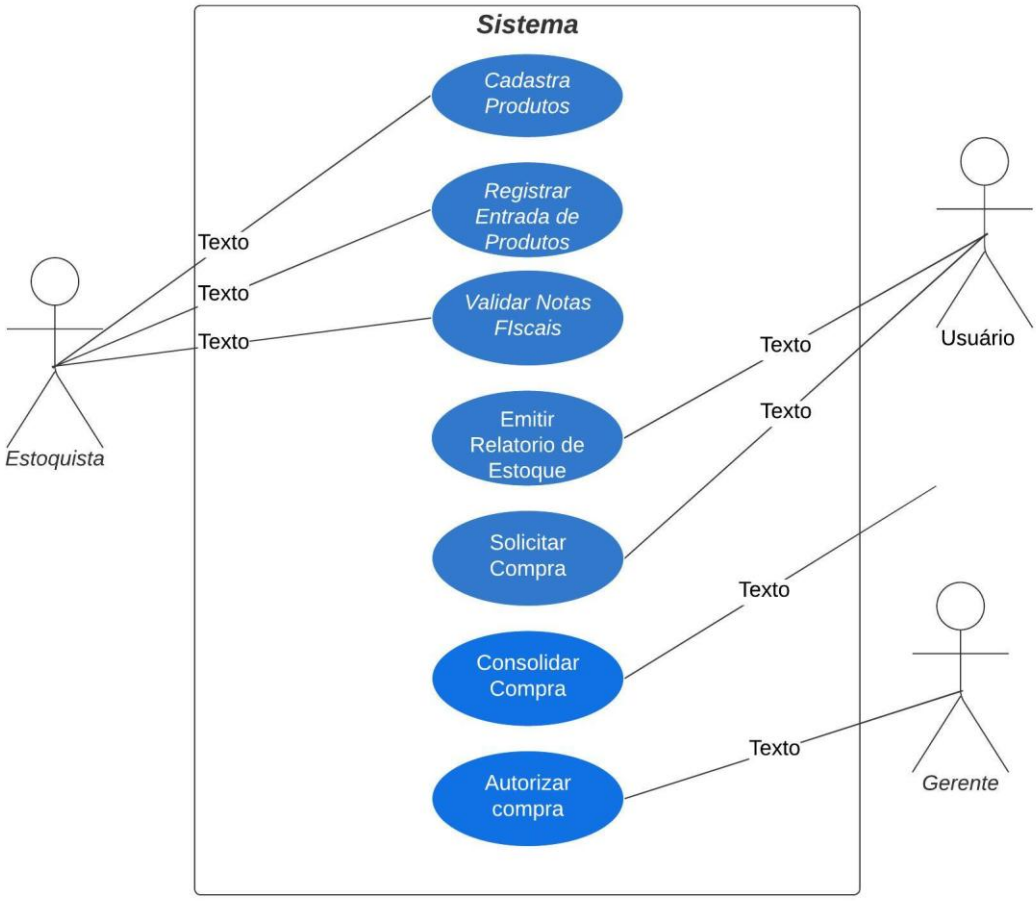
**Atores**

- **Estoquista**: Responsável por registrar a entrada de produtos e validar notas fiscais.
- **Usuário**: Visualiza relatórios semanais e solicita compras de produtos.
- **Gerente de Setor**: Autoriza solicitações de compras de produtos.

**Casos de Uso**

- **Cadastrar Produto**: Estoquista cadastra novos produtos no sistema, incluindo informações como nome, categoria, quantidade e localização.
- **Registrar Entrada de Produto**: Estoquista registra a entrada de novos produtos no estoque, atualizando as quantidades.
- **Validar Nota Fiscal**: Estoquista valida a nota fiscal durante o registro de entrada.
- **Emitir Relatório de Estoque**: Usuário pode gerar relatórios de status do estoque semanalmente.
- **Solicitar Compra de Produto**: Usuário solicita a compra de produtos quando o estoque está baixo.

- **Consolidar Compra**: Processo de consolidação de todas as solicitações de compra para envio ao gerente.
- **Autorizar Compra**: Gerente de Setor analisa as solicitações e autoriza a compra de produtos.



## 3. CONCLUSÃO

O desenvolvimento deste sistema de gerenciamento de estoque possibilitou à empresa um maior controle e eficiência na administração de seus produtos, contribuindo para a redução de problemas como rupturas de estoque e armazenamento excessivo. A implementação da metodologia Scrum facilitou o progresso contínuo e a adaptação do sistema às necessidades do usuário final. Além disso, a modelagem lógica e o diagrama de casos de uso ofereceram uma visão clara e organizada das funcionalidades e interações do sistema. Concluímos que o sistema é eficaz e pode ser expandido com funcionalidades adicionais, como integração com sistemas de compras, para atender às futuras demandas da empresa em crescimento.

## 4. REFERÊNCIAS

CANAL Professor Fabio Guedes. **Criando Kanban Scrum para gestão Ágil de projetos**. Disponível em < https://www.youtube.com/watch?v=_HpsaRL9Jug> Acesso em: 03.nov.2024.

Creately Blog. **Tutorial do diagrama de caso de uso**. Disponível em <https://creately.com/blog/pt/diagrama/tutorial-de-diagrama-de-caso-de-uso/> Acesso em: 03.nov.2024.